

Pythonia



Cuadernillo-Diario

del programador

Conceptos, tips, planillas y plantillas para registrar cada código que escribes.

Este cuaderno pertenece a

Empecé el

Mi meta

Cómo usar este cuaderno

Este cuaderno es tu diario de programación. Los manuales te enseñan; este cuaderno guarda la prueba de lo que aprendiste. Imprímelo completo o solo las plantillas que necesites, tantas veces como quieras.

Antes de empezar, completa el contrato de aprendizaje y el mapa de progreso. **Cada lunes**, llena un planificador semanal. **Cada vez que escribas un programa** que valga la pena, usa una ficha de registro de ejecución: primero escribe qué esperas que salga y después ejecútalo. Comparar lo esperado con lo real es la forma más rápida de entender cómo piensa Python.

Cada vez que apruebes una práctica evaluable en la app, anótala en la bitácora de prácticas: intentos, pistas, el error que más te costó y qué aprendiste. Y marca la pieza en **Avance de mis proyectos**. **Cada error nuevo** va a la bitácora de errores. En pocas semanas tendrás tu propio manual de soluciones. **Cada proyecto** empieza con una plantilla de proyecto, y los fragmentos útiles van a tu biblioteca de snippets.

Pinta el rastreador de racha todos los días que practiques. No buscamos días perfectos, buscamos no cortar la cadena.

Qué hay dentro

Sección	Para qué
Contrato, mapa de progreso, árbol de ramas y racha	Para arrancar y sostener el hábito
Conceptos clave, chuleta, errores comunes y tips de oro	Consulta rápida mientras programas
Planificador semanal (x4)	Organiza tu semana y reflexiona
Avance de mis proyectos	Las piezas de los 4 proyectos del tronco
Registro de ejecución de código (x16)	El corazón del diario
Bitácora de prácticas evaluables (x4)	Lo que aprendiste en cada práctica aprobada
Bitácora de errores (x2)	Tu manual personal de soluciones
Plantilla de proyecto (x2) y biblioteca de snippets (x2)	Para construir y recordar
Páginas de notas con puntos (x4)	Diagramas, ideas y borradores

El ciclo de cada ejecución

1. Predice Escribe qué crees que va a pasar.	2. Ejecuta Corre el código sin cambiar nada.	3. Compara ¿Coincide? Si no, ¿por qué?	4. Anota Registra error, solución y aprendizaje.
--	--	--	--

Mi contrato de aprendizaje

Lo firmas una vez, lo relees cuando flaquees

¿Por qué quiero aprender Python?

Lo que quiero ser capaz de construir en 3 meses

Mi horario fijo de práctica (días y hora)

Cómo voy a celebrar cada nivel completado

Qué haré cuando me trabe (a quién pregunto, dónde busco)

Mis compromisos

- ☐ Practicar al menos 20 minutos, 5 días por semana
- ☐ Escribir el código a mano antes de copiarlo
- ☐ Registrar cada ejecución importante en este cuaderno
- ☐ Anotar cada error nuevo en la bitácora
- ☐ Terminar un proyecto por nivel, aunque no quede perfecto

Firma

Fecha

Mapa de progreso

Marca cada lección al terminarla

Nivel 1: Explorador

Principiante

- ☐ 1. Tu primer programa
- ☐ 2. Variables y tipos de datos
- ☐ 3. Operadores
- ☐ 4. Cadenas de texto (strings)
- ☐ 5. Entrada del usuario y conversiones
- ☐ 6. Condicionales
- ☐ 7. Bucles for y while
- ☐ 8. Listas y tuplas
- ☐ 9. Diccionarios y conjuntos
- ☐ 10. Funciones
- ☐ 11. Errores y cómo leerlos
- ☐ Proyecto: Adivina el número

Nivel 2: Constructor

Intermedio

- ☐ 1. Comprensiones
- ☐ 2. Funciones avanzadas
- ☐ 3. Módulos y paquetes
- ☐ 4. Archivos, CSV y JSON
- ☐ 5. Excepciones a fondo
- ☐ 6. Programación orientada a objetos I
- ☐ 7. Programación orientada a objetos II
- ☐ 8. Entornos virtuales y pip
- ☐ 9. Pruebas con assert y pytest
- ☐ 10. Depuración profesional
- ☐ Proyecto: Gestor de tareas de consola

Nivel 3: Arquitecto

Avanzado

- ☐ 1. Iteradores y generadores
- ☐ 2. Decoradores
- ☐ 3. Context managers
- ☐ 4. Type hints y dataclasses
- ☐ 5. collections, itertools y functools
- ☐ 6. Expresiones regulares
- ☐ 7. Concurrencia y asyncio
- ☐ 8. Logging profesional
- ☐ 9. APIs y HTTP
- ☐ 10. Empaquetado y distribución
- ☐ Proyecto: Monitor asíncrono de precios

Nivel 4: Maestro

Experto

- ☐ 1. El modelo de datos a fondo
- ☐ 2. Descriptores y properties
- ☐ 3. Metaprogramación
- ☐ 4. Internals de CPython
- ☐ 5. Profiling y optimización
- ☐ 6. Concurrencia avanzada
- ☐ 7. Patrones de diseño pythónicos
- ☐ 8. Testing avanzado
- ☐ 9. Rendimiento nativo
- ☐ 10. Arquitectura y calidad
- ☐ Proyecto: Framework de automatización con plugins

Mi árbol de ramas

Test vocacional, ruta principal y Modo Total

Mi top 3 del test

Ruta principal

Ruta secundaria

Temporada 1 (Durante el Nivel 2)

Automatización de oficina	☐ Archivos y carpetas	☐ Excel con openpyxl	☐ Word y PDF
	☐ Correo automático	☐ Automatización web	☐ Tareas programadas y robustez
Videojuegos	☐ El bucle de juego	☐ Sprites y animación	☐ Colisiones
	☐ Física, sonido y eventos	☐ Estados y niveles	☐ Rendimiento y publicación
Bases de datos	☐ SQL esencial	☐ sqlite3 desde Python	☐ Modelado de datos
	☐ SQLAlchemy ORM	☐ PostgreSQL y arquitectura de d...	☐ Análisis con DuckDB

Temporada 2 (Durante el Nivel 3)

Ciencia de datos	☐ NumPy y arrays	☐ pandas: limpieza y transformac...	☐ Agrupaciones, pivotes y fechas
	☐ Visualización	☐ Estadística aplicada	☐ Reportes automáticos
Dashboards y BI	☐ Gráficos interactivos con Plotly	☐ Primera app en Streamlit	☐ Filtros y estado
	☐ Caché y rendimiento	☐ Multi-página y autenticación	☐ Publicación
Desarrollo web	☐ HTTP y la web	☐ Flask: rutas y plantillas	☐ Bases de datos con ORM
	☐ APIs REST con FastAPI	☐ Autenticación y seguridad	☐ Despliegue
Bots y mensajería	☐ Cómo funciona un bot	☐ Comandos y respuestas	☐ Teclados y botones
	☐ Persistencia de datos	☐ Webhooks	☐ Despliegue 24/7

Temporada 3 (Durante el Nivel 4)

Machine learning e IA	☐ Fundamentos y preparación de...	☐ Regresión y clasificación	☐ Evaluación y validación
	☐ Redes neuronales con PyTorch	☐ NLP y transformers	☐ Apps con LLM y RAG
DevOps y scripting	☐ Scripts de sistema	☐ CLIs con Typer	☐ Docker y contenedores
	☐ Automatizar servidores	☐ Nube (AWS/GCP)	☐ Monitoreo
Ciberseguridad defensiva	☐ Fundamentos y ética	☐ Hashing y contraseñas	☐ Cifrado
	☐ Redes y sockets	☐ Análisis de logs	☐ Auditoría de tu propia red
Apps de escritorio	☐ Ventanas y widgets	☐ Layouts	☐ Eventos
	☐ Formularios y validación	☐ Apps con datos	☐ Empaquetar un .exe

Temporada 4 (Posgrado)

IoT y hardware	☐ MicroPython en placas	☐ Sensores y actuadores	☐ Raspberry Pi
	☐ MQTT	☐ Datos en tiempo real	☐ Proyecto conectado
Computación científica	☐ Álgebra simbólica	☐ Cálculo numérico	☐ Optimización
	☐ Simulaciones	☐ Señales	☐ Cuadernos reproducibles

Capstone total: plataforma integral

☐ Bot + API + base de datos + robot + dashboard + modelo, con CI y despliegue

Rastreador de racha

12 semanas: pinta cada día que practicaste

	L	M	X	J	V	S	D
Semana 1							
Semana 2							
Semana 3							
Semana 4							
Semana 5							
Semana 6							
Semana 7							
Semana 8							
Semana 9							
Semana 10							
Semana 11							
Semana 12							

Código de colores sugerido: verde = practiqué, amarillo = repasé, rojo = proyecto, vacío = descanso.

Racha más larga

Días totales

Recompensa

Avance de mis proyectos

Cada práctica aprobada en la app suma una pieza

Nivel 1 • Adivina el número

adivina_el_numero.py

- | | |
|---|--|
| ☐ 1.1 Capítulo 1: la bienvenida | ☐ 1.7 Capítulo 7: la computadora juega |
| ☐ 1.2 Capítulo 2: la configuración | ☐ 1.8 Capítulo 8: el historial |
| ☐ 1.3 Capítulo 3: los cálculos | ☐ 1.9 Capítulo 9: los récords |
| ☐ 1.4 Capítulo 4: los mensajes | ☐ 1.10 Capítulo 10: las funciones del juego |
| ☐ 1.5 Capítulo 5: la entrada del jugador | ☐ 1.11 Capítulo 11: a prueba de errores |
| ☐ 1.6 Capítulo 6: las pistas | |
| ☐ Ejecuté la demo | ☐ Lo descargué y lo abrí en mi IDE |

Lo mejoré con: _____

Nivel 2 • Gestor de tareas de consola

gestor_tareas.py

- | | |
|---|---|
| ☐ 2.1 Filtros con comprensiones | ☐ 2.6 La clase Tarea |
| ☐ 2.2 Creación y orden de tareas | ☐ 2.7 Herencia: TareaConFecha |
| ☐ 2.3 Resumen y estadísticas | ☐ 2.8 Lector de requirements |
| ☐ 2.4 Persistencia en JSON | ☐ 2.9 Tests de la clase Tarea |
| ☐ 2.5 Validación con excepciones | ☐ 2.10 Orden y conteo depurados |
| ☐ Ejecuté la demo | ☐ Lo descargué y lo abrí en mi IDE |

Lo mejoré con: _____

Nivel 3 • Monitor de precios

monitor_precios.py

- | | |
|---|--|
| ☐ 3.1 Lectura perezosa con generadores | ☐ 3.6 Extracción con regex |
| ☐ 3.2 Decoradores de robustez | ☐ 3.7 Planificación de consultas concurrentes |
| ☐ 3.3 Context managers | ☐ 3.8 Registro de cambios con niveles |
| ☐ 3.4 Modelos de datos | ☐ 3.9 Cliente de API |
| ☐ 3.5 Análisis con collections | ☐ 3.10 Versionado del monitor |
| ☐ Ejecuté la demo | ☐ Lo descargué y lo abrí en mi IDE |

Lo mejoré con: _____

Nivel 4 • Framework de automatización con plugins

mini_framework.py

- | | |
|---|--|
| ☐ 4.1 Pipeline (modelo de datos) | ☐ 4.6 Reparto de trabajo |
| ☐ 4.2 Config con descriptores | ☐ 4.7 Bus de eventos |
| ☐ 4.3 Registro automático de plugins | ☐ 4.8 Tests del ejecutor de tareas |
| ☐ 4.4 Utilidades del motor | ☐ 4.9 Cálculo vectorial |
| ☐ 4.5 Utilidades optimizadas | ☐ 4.10 Aplicación y raíz de composición |
| ☐ Ejecuté la demo | ☐ Lo descargué y lo abrí en mi IDE |

Lo mejoré con: _____

Conceptos clave

Concepto	Significado
Algoritmo	Secuencia de pasos para resolver un problema.
Argumento	Valor que se pasa a una función al llamarla.
Bucle	Estructura que repite un bloque de código (for, while).
Clase	Molde para crear objetos con datos y comportamiento.
Comprensión	Sintaxis compacta para crear listas, dicts o sets.
Context manager	Objeto que prepara y limpia recursos en un bloque with.
Corrutina	Función async que puede pausarse con await.
Decorador	Función que envuelve a otra para extender su comportamiento.
Diccionario	Colección de pares clave-valor.
Docstring	Texto que documenta un módulo, clase o función.
Entorno virtual	Instalación aislada de Python por proyecto.
Excepción	Señal de que ocurrió un error durante la ejecución.
f-string	Texto con variables insertadas: f"Hola {nombre}".
Función	Bloque de código reutilizable con nombre.
Generador	Función con yield que produce valores bajo demanda.
GIL	Bloqueo que permite a un solo hilo ejecutar bytecode a la vez en CPython.
Herencia	Mecanismo por el que una clase reutiliza otra.
Indentación	Sangría que define los bloques de código en Python.
Inmutable	Objeto que no puede modificarse tras crearse (str, tuple).
Iterable	Objeto que se puede recorrer con for.
Lambda	Función anónima de una sola expresión.
Lista	Colección ordenada y mutable.
Método	Función que pertenece a una clase.
Módulo	Archivo .py que se puede importar.
Mutable	Objeto que puede modificarse (list, dict, set).
Objeto	Instancia concreta de una clase.
Parámetro	Nombre que recibe un valor en la definición de una función.
PEP 8	Guía oficial de estilo para código Python.
pip	Instalador de paquetes de Python.
Scope	Ámbito donde un nombre es visible (regla LEGB).
Set	Colección de elementos únicos sin orden.
Slicing	Extraer partes de una secuencia: s[inicio:fin:paso].
Traceback	Informe del recorrido que llevó a un error.
Tupla	Colección ordenada e inmutable.

Concepto	Significado
Type hint	Anotación que indica el tipo esperado de un valor.
Variable	Nombre que apunta a un valor en memoria.

Chuleta de Python

Tema	Lo esencial
Texto (str)	<code>.upper()</code> <code>.lower()</code> <code>.strip()</code> <code>.split(",")</code> <code>.replace(a, b)</code> <code>.startswith(x)</code> <code>.find(x)</code> <code>" , ".join(lista)</code> <code>f"{x:.2f}"</code>
Listas	<code>.append(x)</code> <code>.insert(i, x)</code> <code>.pop()</code> <code>.remove(x)</code> <code>.sort()</code> <code>sorted(l)</code> <code>len(l)</code> <code>l[::-1]</code> <code>x in l</code> <code>enumerate(l)</code> <code>zip(a, b)</code>
Diccionarios	<code>d[k]</code> <code>d.get(k, defecto)</code> <code>d.items()</code> <code>d.keys()</code> <code>d.values()</code> <code>d.update(otro)</code> <code>d.pop(k)</code> <code>d.setdefault(k, [])</code> <code>{**a, **b}</code>
Conjuntos	<code>a b</code> (unión) <code>a & b</code> (intersección) <code>a - b</code> (diferencia) <code>.add(x)</code> <code>.discard(x)</code> <code>set(lista)</code> para quitar duplicados
Control de flujo	<code>if / elif / else</code> <code>x if cond else y</code> <code>for x in iterable:</code> <code>while cond:</code> <code>break</code> <code>continue</code> <code>match valor:</code> <code>case 1:</code> <code>...</code> <code>case _:</code> <code>...</code>
Funciones	<code>def f(a, b=0, *args, **kwargs) -> tipo:</code> <code>"""docstring"""</code> <code>return resultado</code> <code>lambda x: x * 2</code>
Errores	<code>try: ... except ValueError as e: ...</code> <code>else: ... finally: ... raise MiError("msg")</code>
Archivos	<code>from pathlib import Path</code> <code>with open(ruta, encoding="utf-8") as f: texto = f.read()</code> <code>Path("a.txt").write_text("hola", encoding="utf-8")</code>
f-strings	<code>f"{n:,}"</code> miles <code>f"{x:.2f}"</code> decimales <code>f"{p:.1%}"</code> porcentaje <code>f"{t:>10}"</code> alinear <code>f"{fecha:%d/%m/%Y}"</code> <code>f"{var=}"</code> depurar

Errores comunes y cómo resolverlos

Error	Causa típica	Solución
<code>SyntaxError</code>	Falta <code>:</code> , paréntesis o comillas	Revisa la línea indicada y la anterior
<code>IndentationError</code>	Sangría inconsistente	Usa siempre 4 espacios, nunca mezcles tabs
<code>NameError</code>	Variable no definida o mal escrita	Revisa ortografía y orden de definición
<code>TypeError</code>	Operación entre tipos incompatibles	Convierte con <code>int()</code> , <code>str()</code> , <code>float()</code>
<code>ValueError</code>	Valor con tipo correcto pero inválido	Valida la entrada antes de convertir
<code>IndexError</code>	Índice fuera del rango de la lista	Verifica <code>len()</code> y recuerda que empieza en 0
<code>KeyError</code>	La clave no existe en el diccionario	Usa <code>d.get(clave, defecto)</code> o <code>'in'</code>
<code>AttributeError</code>	El objeto no tiene ese atributo/método	Revisa el tipo con <code>type()</code> y <code>dir()</code>
<code>ZeroDivisionError</code>	División entre cero	Valida el divisor antes de dividir
<code>FileNotFoundError</code>	La ruta del archivo no existe	Usa <code>pathlib</code> y verifica con <code>.exists()</code>
<code>ModuleNotFoundError</code>	Librería no instalada o entorno incorrecto	Activa el <code>venv</code> y usa <code>pip install</code>
<code>RecursionError</code>	Recursión sin caso base	Asegura una condición de salida

20 tips de oro

1

Lee el error completo antes de buscarlo en internet: la última línea dice qué pasó.

2

Escribe código todos los días. La constancia gana a la intensidad.

3

Nombra las cosas por lo que son: `total_ventas` dice más que `tv`.

4

Una función, una responsabilidad.

5

Primero que funcione, después que sea bonito, al final que sea rápido.

6

Prueba los casos borde: vacío, cero, negativo, gigante, con tildes.

7

Guarda versiones con Git desde el primer día.

8

Si copias y pegas código tres veces, conviértelo en una función.

9

Explica tu código en voz alta: encontrarás errores solo.

10

Lee código de otras personas; es la forma más rápida de aprender estilos.

11

No memorices todo: aprende a buscar en la documentación oficial (docs.python.org).

12

Descansa: muchos bugs se resuelven después de una caminata.

13

Haz commits pequeños con mensajes claros.

14

Escribe el test antes de arreglar un bug: así no vuelve.

15

Usa el REPL o Jupyter para experimentar antes de escribir el programa.

16

Comenta el porqué, no el qué: el código ya dice qué hace.

17

Configura tu editor con un linter (ruff) desde el inicio.

18

Divide los problemas grandes en pasos tan pequeños que parezcan triviales.

19

Construye proyectos propios: es donde realmente se aprende.

20

Equivocarse es parte del proceso. Cada error anotado es una lección ganada.

Planificador semanal #1

Planifica el lunes, revisa el domingo

Semana del

Nivel / lecciones

Horas objetivo

Objetivo principal de la semana

Día	Tema o lección	Ejercicio / código	Min	Hecho
Lunes				☐
Martes				☐
Miércoles				☐
Jueves				☐
Viernes				☐
Sábado				☐
Domingo				☐

Reflexión del domingo

Lo que salió bien

Lo que me costó

Concepto que debo repasar

Ajuste para la próxima semana

Constancia ○ ○ ○ ○ ○

Comprensión ○ ○ ○ ○ ○

Planificador semanal #2

Planifica el lunes, revisa el domingo

Semana del

Nivel / lecciones

Horas objetivo

Objetivo principal de la semana

Día	Tema o lección	Ejercicio / código	Min	Hecho
Lunes				☐
Martes				☐
Miércoles				☐
Jueves				☐
Viernes				☐
Sábado				☐
Domingo				☐

Reflexión del domingo

Lo que salió bien

Lo que me costó

Concepto que debo repasar

Ajuste para la próxima semana

Constancia ○ ○ ○ ○ ○

Comprensión ○ ○ ○ ○ ○

Planificador semanal #3

Planifica el lunes, revisa el domingo

Semana del

Nivel / lecciones

Horas objetivo

Objetivo principal de la semana

Día	Tema o lección	Ejercicio / código	Min	Hecho
Lunes				☐
Martes				☐
Miércoles				☐
Jueves				☐
Viernes				☐
Sábado				☐
Domingo				☐

Reflexión del domingo

Lo que salió bien

Lo que me costó

Concepto que debo repasar

Ajuste para la próxima semana

Constancia ○ ○ ○ ○ ○

Comprensión ○ ○ ○ ○ ○

Planificador semanal #4

Planifica el lunes, revisa el domingo

Semana del

Nivel / lecciones

Horas objetivo

Objetivo principal de la semana

Día	Tema o lección	Ejercicio / código	Min	Hecho
Lunes				☐
Martes				☐
Miércoles				☐
Jueves				☐
Viernes				☐
Sábado				☐
Domingo				☐

Reflexión del domingo

Lo que salió bien

Lo que me costó

Concepto que debo repasar

Ajuste para la próxima semana

Constancia ○ ○ ○ ○ ○

Comprensión ○ ○ ○ ○ ○

Registro de ejecución #01

Escribe, ejecuta, compara, aprende

Fecha

Nivel / lección / reto

Tiempo (min)

¿Qué quiero que haga este código?

Mi código

1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21

Salida que espero

Salida real

Resultado

☐ Funcionó a la primera ☐ Funcionó tras corregir ☐ Aún no funciona

Error o mensaje que apareció

Cómo lo resolví

Lo que aprendí

Dificultad ○ ○ ○ ○ ○ Confianza ○ ○ ○ ○ ○ ☐ Commit en Git

Registro de ejecución #02

Escribe, ejecuta, compara, aprende

Fecha

Nivel / lección / reto

Tiempo (min)

¿Qué quiero que haga este código?

Mi código

1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21

Salida que espero

Salida real

Resultado

☐ Funcionó a la primera ☐ Funcionó tras corregir ☐ Aún no funciona

Error o mensaje que apareció

Cómo lo resolví

Lo que aprendí

Dificultad ○ ○ ○ ○ ○ Confianza ○ ○ ○ ○ ○ ☐ Commit en Git

Registro de ejecución #03

Escribe, ejecuta, compara, aprende

Fecha

Nivel / lección / reto

Tiempo (min)

¿Qué quiero que haga este código?

Mi código

1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21

Salida que espero

Salida real

Resultado

☐ Funcionó a la primera ☐ Funcionó tras corregir ☐ Aún no funciona

Error o mensaje que apareció

Cómo lo resolví

Lo que aprendí

Dificultad ○ ○ ○ ○ ○ Confianza ○ ○ ○ ○ ○ ☐ Commit en Git

Registro de ejecución #04

Escribe, ejecuta, compara, aprende

Fecha

Nivel / lección / reto

Tiempo (min)

¿Qué quiero que haga este código?

Mi código

1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21

Salida que espero

Salida real

Resultado

☐ Funcionó a la primera ☐ Funcionó tras corregir ☐ Aún no funciona

Error o mensaje que apareció

Cómo lo resolví

Lo que aprendí

Dificultad ○ ○ ○ ○ ○ Confianza ○ ○ ○ ○ ○ ☐ Commit en Git

Registro de ejecución #05

Escribe, ejecuta, compara, aprende

Fecha

Nivel / lección / reto

Tiempo (min)

¿Qué quiero que haga este código?

Mi código

1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21

Salida que espero

Salida real

Resultado

☐ Funcionó a la primera ☐ Funcionó tras corregir ☐ Aún no funciona

Error o mensaje que apareció

Cómo lo resolví

Lo que aprendí

Dificultad ○ ○ ○ ○ ○ Confianza ○ ○ ○ ○ ○ ☐ Commit en Git

Registro de ejecución #06

Escribe, ejecuta, compara, aprende

Fecha

Nivel / lección / reto

Tiempo (min)

¿Qué quiero que haga este código?

Mi código

1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21

Salida que espero

Salida real

Resultado

☐ Funcionó a la primera ☐ Funcionó tras corregir ☐ Aún no funciona

Error o mensaje que apareció

Cómo lo resolví

Lo que aprendí

Dificultad ○ ○ ○ ○ ○ Confianza ○ ○ ○ ○ ○ ☐ Commit en Git

Registro de ejecución #07

Escribe, ejecuta, compara, aprende

Fecha

Nivel / lección / reto

Tiempo (min)

¿Qué quiero que haga este código?

Mi código

1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21

Salida que espero

Salida real

Resultado

☐ Funcionó a la primera ☐ Funcionó tras corregir ☐ Aún no funciona

Error o mensaje que apareció

Cómo lo resolví

Lo que aprendí

Dificultad ○ ○ ○ ○ ○ Confianza ○ ○ ○ ○ ○ ☐ Commit en Git

Registro de ejecución #08

Escribe, ejecuta, compara, aprende

Fecha

Nivel / lección / reto

Tiempo (min)

¿Qué quiero que haga este código?

Mi código

1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21

Salida que espero

Salida real

Resultado

☐ Funcionó a la primera ☐ Funcionó tras corregir ☐ Aún no funciona

Error o mensaje que apareció

Cómo lo resolví

Lo que aprendí

Dificultad ○ ○ ○ ○ ○ Confianza ○ ○ ○ ○ ○ ☐ Commit en Git

Registro de ejecución #09

Escribe, ejecuta, compara, aprende

Fecha

Nivel / lección / reto

Tiempo (min)

¿Qué quiero que haga este código?

Mi código

1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21

Salida que espero

Salida real

Resultado

☐ Funcionó a la primera ☐ Funcionó tras corregir ☐ Aún no funciona

Error o mensaje que apareció

Cómo lo resolví

Lo que aprendí

Dificultad ○ ○ ○ ○ ○ Confianza ○ ○ ○ ○ ○ ☐ Commit en Git

Registro de ejecución #10

Escribe, ejecuta, compara, aprende

Fecha

Nivel / lección / reto

Tiempo (min)

¿Qué quiero que haga este código?

Mi código

1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21

Salida que espero

Salida real

Resultado

☐ Funcionó a la primera ☐ Funcionó tras corregir ☐ Aún no funciona

Error o mensaje que apareció

Cómo lo resolví

Lo que aprendí

Dificultad ○ ○ ○ ○ ○ Confianza ○ ○ ○ ○ ○ ☐ Commit en Git

Registro de ejecución #11

Escribe, ejecuta, compara, aprende

Fecha

Nivel / lección / reto

Tiempo (min)

¿Qué quiero que haga este código?

Mi código

1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21

Salida que espero

Salida real

Resultado

☐ Funcionó a la primera ☐ Funcionó tras corregir ☐ Aún no funciona

Error o mensaje que apareció

Cómo lo resolví

Lo que aprendí

Dificultad ☐ ☐ ☐ ☐ ☐ Confianza ☐ ☐ ☐ ☐ ☐ ☐ Commit en Git

Registro de ejecución #12

Escribe, ejecuta, compara, aprende

Fecha

Nivel / lección / reto

Tiempo (min)

¿Qué quiero que haga este código?

Mi código

1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21

Salida que espero

Salida real

Resultado

☐ Funcionó a la primera ☐ Funcionó tras corregir ☐ Aún no funciona

Error o mensaje que apareció

Cómo lo resolví

Lo que aprendí

Dificultad ○ ○ ○ ○ ○ Confianza ○ ○ ○ ○ ○ ☐ Commit en Git

Registro de ejecución #13

Escribe, ejecuta, compara, aprende

Fecha

Nivel / lección / reto

Tiempo (min)

¿Qué quiero que haga este código?

Mi código

1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21

Salida que espero

Salida real

Resultado

☐ Funcionó a la primera ☐ Funcionó tras corregir ☐ Aún no funciona

Error o mensaje que apareció

Cómo lo resolví

Lo que aprendí

Dificultad ○ ○ ○ ○ ○ Confianza ○ ○ ○ ○ ○ ☐ Commit en Git

Registro de ejecución #14

Escribe, ejecuta, compara, aprende

Fecha

Nivel / lección / reto

Tiempo (min)

¿Qué quiero que haga este código?

Mi código

1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21

Salida que espero

Salida real

Resultado

☐ Funcionó a la primera ☐ Funcionó tras corregir ☐ Aún no funciona

Error o mensaje que apareció

Cómo lo resolví

Lo que aprendí

Dificultad ○ ○ ○ ○ ○ Confianza ○ ○ ○ ○ ○ ☐ Commit en Git

Registro de ejecución #15

Escribe, ejecuta, compara, aprende

Fecha

Nivel / lección / reto

Tiempo (min)

¿Qué quiero que haga este código?

Mi código

1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21

Salida que espero

Salida real

Resultado

☐ Funcionó a la primera ☐ Funcionó tras corregir ☐ Aún no funciona

Error o mensaje que apareció

Cómo lo resolví

Lo que aprendí

Dificultad ☐ ☐ ☐ ☐ ☐ Confianza ☐ ☐ ☐ ☐ ☐ ☐ Commit en Git

Registro de ejecución #16

Escribe, ejecuta, compara, aprende

Fecha

Nivel / lección / reto

Tiempo (min)

¿Qué quiero que haga este código?

Mi código

1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21

Salida que espero

Salida real

Resultado

☐ Funcionó a la primera ☐ Funcionó tras corregir ☐ Aún no funciona

Error o mensaje que apareció

Cómo lo resolví

Lo que aprendí

Dificultad ○ ○ ○ ○ ○ Confianza ○ ○ ○ ○ ○ ☐ Commit en Git

Bitácora de prácticas #1

Cada práctica aprobada, contada con tus palabras

Bitácora de prácticas #2

Cada práctica aprobada, contada con tus palabras

Bitácora de prácticas #3

Cada práctica aprobada, contada con tus palabras

Bitácora de prácticas #4

Cada práctica aprobada, contada con tus palabras

Bitácora de errores #1

Cada error anotado es una lección ganada

Bitácora de errores #2

Cada error anotado es una lección ganada

Plantilla de proyecto #1

Del problema al programa terminado

Nombre del proyecto

Inicio / entrega

¿Qué problema resuelve y para quién?

Entradas (qué recibe)	Salidas (qué produce)

- Funcionalidades (marca al completar)
- ☐
- ☐
- ☐
- ☐
- ☐
- ☐

Esquema: módulos, clases, funciones o flujo

Librerías que usaré

Casos de prueba

Retrospectiva: qué mejoraría en la versión 2

Plantilla de proyecto #2

Del problema al programa terminado

Nombre del proyecto

Inicio / entrega

¿Qué problema resuelve y para quién?

Entradas (qué recibe)	Salidas (qué produce)

- Funcionalidades (marca al completar)
- ☐
- ☐
- ☐
- ☐
- ☐
- ☐

Esquema: módulos, clases, funciones o flujo

Librerías que usaré

Casos de prueba

Retrospectiva: qué mejoraría en la versión 2

Mi biblioteca de snippets #1

Fragmentos que quiero recordar

Para qué sirve

Etiquetas

Código

Para qué sirve

Etiquetas

Código

Para qué sirve

Etiquetas

Código

Para qué sirve

Etiquetas

Código

Mi biblioteca de snippets #2

Fragmentos que quiero recordar

Para qué sirve

Etiquetas

Código

Para qué sirve

Etiquetas

Código

Para qué sirve

Etiquetas

Código

Para qué sirve

Etiquetas

Código

